DESIGN CONTRACT · REV 2 · FLEET LIFECYCLE

Fleet Provisioning, Build Management & LSV Lifecycle

A single pipeline from bare metal to a continuously-patched host — provision, enroll, configure by function, maintain on latest-stable, and retire cleanly — built as increments on the machinery SolariNet already has, not a rewrite.

Target: **dashboard + benzene control node** | Engine: **Ansible + SoR** | **Rev 2 · folds in prior design review**

··· Final feature before stability freeze

WHAT CHANGED IN REV 2

This revision incorporates a prior design review (GPT-5.5, checked against the repository). It corrects three overstated claims, resolves five blockers into concrete design — host static-IP cutover, a single enrollment protocol with real endpoints, atomic IP allocation, the true xenon→benzene execution topology, and a de-circularised roadmap — and adds lifecycle edges omitted from the first draft. An independent cross-lab implementation review remains a release gate.

01 Overview & architectural stance

What we are building, and the one decision that shapes everything downstream.

SolariNet can already stage a PXE install, mint an enrollment certificate, mirror hosts into a system-of-record, and push the agent to a running box. What it *cannot* do is run any of that as one motion, decide where a host belongs from its *function*, or keep the fleet current over time. This plan closes those gaps and unifies the pieces into a lifecycle — provision, configure, maintain, and retire.

LOAD-BEARING DECISION

The dashboard orchestrates and observes; benzene executes. The PHP UI never SSHes into a host, never runs a playbook, and holds no CA key. It *enqueues intent* and *reads back state*. Concretely: the dashboard calls a `solariCtl` verb on **xenon** (where solariServer and the bridge actually run), which writes an immutable `run` record into the SoR and signals the message bus. A **run-queue daemon on benzene** consumes that record and drives Ansible. This is the same transactional-outbox pattern the `sorsync` daemons already use — and it means a compromised dashboard can enqueue a request but cannot itself touch a host or sign a cert.

Three principles follow and hold throughout:

- **The SoR is the single source of truth** — and therefore *is* the Ansible inventory. Function tags live in the CMDB and drive inventory groups, IP allocation, package bundles, and config. There is no second inventory to keep in sync.
- **Function, not hostname, is the unit of intent.** You tag a host `database` or `auth`; the system derives its subnet, package set, DNS name, and config from that tag.
- **Every layer is an increment.** Each phase wires together something that already half-exists. We have two disconnected provisioning flows today; the first job is to make them one, not to add a third.

SCOPE BOUNDARY SET IN REV 2

The covered footprint is **glibc Linux servers** (Debian / Ubuntu / openSUSE × x86_64 / arm64 / arm32) on **IPv4**. Musl/mips appliances wait on the static-musl build matrix (a pre-existing blocker); IPv6 allocation waits on address-family-aware DNS rendering. Both are named follow-ons, not hidden assumptions.

02 Ground truth — what already exists

Verified against the repository by three read-only investigators and an adversarial review.
`OBSERVED` = seen in code; `INFERRED` = reasoned; `NEW` = to build; `CORRECTED` = Rev-1 claim fixed after code check.

Much foundation is in place — the problem is that the pieces are *disconnected*, each with its own enrollment story, none wired to the dashboard or to each other.

CAPABILITY	STATE	WHERE (cited)
PXE / netboot staging	STAGED / INERT	`deploy/fleet/fleet-provision.sh`, `netboot/`. Renders per-MAC iPXE + unattended config, pushes to benzene. Live boot needs the DHCP switch thrown; non-destructive today.
Ansible fleet tree	MATURE, UNWIRED	`deploy/fleet/ansible/`. Real roles & a shared cross-OS package contract — run manually, inventory is placeholder `192.0.2.x`, called by nothing.
SoR / CMDB	LIVE	`netdb/sor/schema.sql` on Cesium. The 2026-10-22 live parity check found 45 base tables, four views, and 24 triggers on both Cesium and Benzene. Migration 021 adds the function taxonomy; `entities.role` remains descriptive free text.
DNS from SoR (A + PTR, IPv4)	AUTOMATIC	`v_dns_hosts` view → CDC → `sor_apply_dnsd.py`. A live entity + `is_primary` IP renders forward & reverse — but `gen-zones.py` emits only `A` and `/24 in-addr.arpa` (IPv4-shaped, see §7).
Remote agent deploy	WORKS	`deploy/remote-deploy.sh` — adaptive SSH push, signs a per-host cert. Consumes existing SSH access; does not provision it.

CAPABILITY	STATE	WHERE (cited)
Config apply / converge	LIVE	`nodeConfig` target-epoch vs applied-epoch. Config only — not packages or OS.
Retire / decommission verbs	EXIST	`src/server/solariCtl.c` already exposes `DECOMMISSION` / `RETIRE`; DNS excludes retired rows. Not surfaced in a provisioning lifecycle (see §8.6).
UniFi → SoR network import	CANDIDATE — NOT LIVE	`deploy/unifi/unifi_network_sor.py` validates paginated controller snapshots and imports them transactionally; scratch integration tests cover idempotence, source conflicts, stale-row deletion, and CDC. Candidate migration 026 adds imported network fact CDC but is not on the live SoR. A read-only live dry-run passed on 2026-10-22 using the trusted Let's Encrypt certificate and a process-only connection pin to 10.0.0.1; it made no SoR changes. Protected runtime configuration and an approved, reviewed SoR import remain.
IP-by-function allocation	CANDIDATE — NOT LIVE	`netdb/sor/allocator.py` and the Provisioning API implement the default-off approval gate, subnet-lock recheck, overlap guard, and reserved-safe allocation. Scratch concurrency and Chromium acceptance pass; candidate migration 025 and the runtime integration are not deployed on Cesium.
DHCP reservation from SoR	ABSENT	Appliers exist for DNS, AD, Pi-hole — none for UniFi/DHCP.
Update / patch / LSV rollout	ABSENT	Only OS-native `unattended-upgrades` (install-only, off by default). No version tracking, orchestration, or schedule.
Static-musl build matrix	BLOCKER	Flagged across three docs. Blocks agent coverage of musl/mips appliances regardless of orchestration.

THE CORE DISCONNECT

Netboot's firstboot enrollment hook is a *stub*; Ansible's `solarinet-client` role uses a *different* mechanism (a pre-staged, control-node-held private key). Neither calls the other, and the two **contradict on where the private key is born** — a contradiction Rev 2 resolves in §8.2. There is no path today from "bare metal boots" → "enrolled in the SoR" → "Ansible converges." Building that single spine is Phase 1.

03 Gap analysis

Everything the requested system needs that the repo does not yet provide, sized honestly.

G1 · IP-by-function allocator

Atomic next-free-address allocation honoring reserved/gateway/broadcast and `dhcp_ranges` boundaries, with DB-enforced live uniqueness. Greenfield (§6).

G2 · Function taxonomy

Closed function-tag set → subnet, package bundle, config profile, criticality. New tables; the semantic spine (§5).

G3 · Unify the provision spine

One enrollment protocol; firstboot → enroll → SoR → allocate → converge as one motion (§8.1–8.3).

G4 · SoR → Ansible inventory

Dynamic inventory from the SoR: tags become groups, `host_vars` from the entity row + an immutable snapshot per run (§8.3, §11).

G5 · LSV lifecycle

Version tracking, immutable per-distro manifests, ringed rollout with health gates, dry-run, rollback/roll-forward, schedule (§8.4). Largest new capability.

G6 · Host network cutover

Configure the allocated static IP *on the host*, per distro, idempotently, with connectivity check + DHCP fallback. The gap that makes allocation real (§8.1).

G7 · UniFi network import

A tested import/reconcile of UniFi VLANs/subnets/pools into the SoR — a **prerequisite** to safe allocation, not an assumption (§6).

G8 · SoR → DHCP applier

New `sor_apply_unifi.py` pushing reservations — or a deliberate static+DNS-only stance with lease quarantine (§7).

G9 · Run-queue & state machine

Normalized run / attempt / target records, leases, per-host mutex, resumable workflow — not an idempotent renderer (§10, §11).

G10 · Lifecycle edges

Retirement/decommission and function re-tagging with real transition workflows (§8.6, §8.7).

G11 · Supply chain & secrets

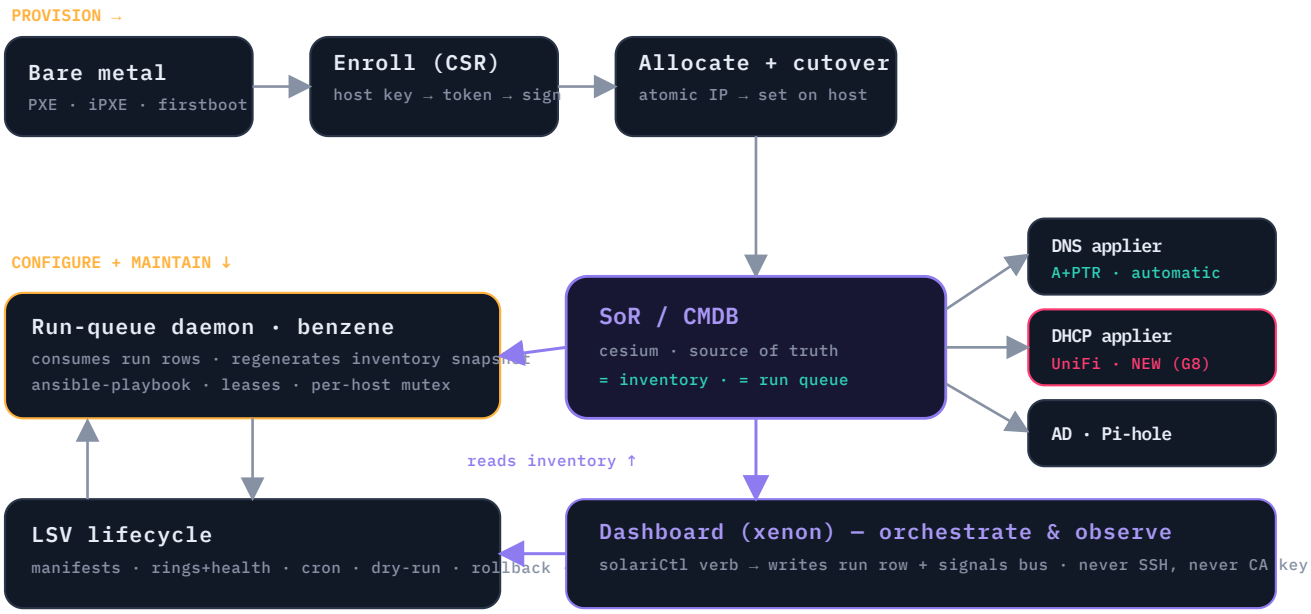
Signed artifact manifests, verified transport, vault identity + secret rotation, log redaction (§12).

G12 · Dashboard surface

Provisioning · Build Management · Updates pages. Orchestrate-and-observe only (§9).

04 Target architecture

Five layers over one source of truth. Control writes intent on xenon; execution consumes it on benzene.



The SoR is the hub and the queue: control writes an immutable run row on xenon; the benzene daemon consumes it, snapshots inventory, and executes. DNS/DHCP/AD fall out of the SoR. Amber = execution (benzene); violet = control plane (xenon dashboard).

Layer 1
Inventory

Source of truth. Function taxonomy + host tags in the SoR; every downstream value derived from a tag, never entered twice.

new: function tables · builds on: entities, ip_addresses

Layer 2
Provision

Bare metal → enrolled. PXE install, one enrollment protocol, atomic IP allocation, host network cutover, SoR registration — DNS falls out.

new: allocator, enroll endpoints, cutover · builds on: fleet-provision.sh, solari-enroll.sh

Layer 3
Configure

Baseline + role. Ansible applies baseline + the tag's role bundle from an immutable inventory snapshot.

new: dynamic inventory, per-tag roles · builds on: ansible/roles/* (after trust refactor)

Layer 4
Maintain

LSV lifecycle. Track installed-vs-manifest, roll in health-gated rings on a schedule, dry-run, roll back / forward.

new: the whole engine

Layer 5
Observe + edges

Drive, watch, retire, re-tag. Three pages enqueue intent and render history; retirement and re-tagging are first-class transitions, not afterthoughts.

new: 2 pages, retire/re-tag flows · builds on: screens3.jsx, solariCtl RETIRE

05 Function taxonomy — the semantic spine

The one new concept that makes "tag a host and everything follows" possible.

Today `entities.role` is a free-text sentence. We introduce a closed set of **function tags**. A host may carry several; each binds which network it belongs on, what software it runs, how it is configured, and its default criticality.

FUNCTION TAG	HOME NETWORK	PACKAGE BUNDLE (LSV)	CONFIG PROFILE
`database`	production	mariadb-server, backup agent	data dir, tuned innodb, backup cron
`auth`	core / security	keycloak or sssd, chrony	realm join, TLS from chlorine CA
`web`	production	apache/php-fpm, certbot	vhost skeleton, ACME
`security`	security VLAN	suricata/zeek, log shipper	span/mirror, retention
`dns`	netmgmt	bind9 or technitium	zone pull from SoR
`monitoring`	core	solariMonitor, snmp	poll targets, MQ creds
`base`	by explicit subnet	core-stack only	baseline hardening only

Illustrative rows — the exact set is a Phase-0 deliverable. Bundles reuse the existing `common-stack` "one logical list + per-OS name map" pattern, extended so each tag contributes its bundle atop the shared core.

MULTI-TAG PRECEDENCE — DECIDED UP FRONT

A host may carry several tags, so the taxonomy defines **exactly one rule** for conflicts: a `primary_function` per host selects the home network (never ambiguous), while package bundles *union* and config profiles apply in a declared tag order. Tag pairs that must never coexist (e.g. `security` + `database` on one box) are declared incompatible and rejected at tag time, not discovered at converge time.

06 IP allocation by function

The first logic to ever *allocate* from the network hierarchy — which means it must first be able to trust that hierarchy.

> **PREREQUISITE (G7) — RESOLVED FROM A REV-1 ERROR**
>
> Rev 1 assumed subnets and `dhcp_ranges` were "seeded from UniFi." They are not — nothing populates `dhcp_ranges`, and subnets are synthesized `/24`s. **The allocator must not run against inferred ranges with no pool boundaries.** So a tested **UniFi network import** (VLANs, subnets, gateways, DHCP pools → SoR, with provenance + freshness) is a hard prerequisite, landed in Phase 0.

Given a trustworthy hierarchy, the allocator turns a function tag into a concrete static address — and does so **atomically**, because the schema does *not* enforce live uniqueness on its own (`UNIQUE(address, deleted_at)` permits many `NULL` rows; `seed.py` says as much):

```
allocateAddress(primaryTag, hostName):                # IPv4 only in this phase
  subnet = subnetForNetwork(networkFor(primaryTag))   # taxonomy → one network → its subnet
  reserved = { network_addr, broadcast_addr, subnet.gateway_ip }
  BEGIN TRANSACTION                                   # serialize concurrent allocators
    lock the subnet's allocation row (SELECT … FOR UPDATE)
    used  = active addresses in subnet (via the enforced live-address key)
    pools = dhcp_ranges WHERE subnet_id = subnet.id   # dynamic_pool + excluded spans
    cand  = first usable host address in subnet.cidr
            NOT IN used  AND NOT IN reserved
            AND NOT inside any dynamic_pool / excluded range
    INSERT entities(...); INSERT ip_addresses(cand, is_primary, assignment='static', …)
  COMMIT                                               # unique index rejects any racing dup
  return cand
```

Two correctness requirements the review surfaced, both adopted:

- **Enforceable live uniqueness** — add a generated *active-address* column (address where not-deleted, else NULL is disallowed) with a real `UNIQUE` index, so the database itself rejects a double-allocation even under a race. The transaction lock makes the common path clean; the index is the backstop.
- **Reserved-address rules** — never hand out the network, broadcast, or gateway address, never anything inside a DHCP `dynamic_pool` / `excluded` span, and respect `static_block` semantics. A wrong role→subnet map is caught by validating the map and showing every first-run allocation for confirmation before commit.

Writing the entity + `is_primary` IP is enough for DNS (§7): CDC fires, `sor_emitd` publishes, `sor_apply_dnsd` renders A + PTR. Writes follow the existing `Sor::upsertHost()` pattern under a new `provisioner` row in `sources` for provenance. IPv6 allocation is deferred until DNS rendering is address-family-aware.

07 DNS & DHCP integration

One side is nearly free — within limits. The other is a real decision with a guardrail.

DNS — automatic, IPv4-shaped OBSERVED

A live `entities` row + a live `is_primary` IP is sufficient: `v_dns_hosts` selects it, CDC emits, `sor_apply_dnsd` renders A + PTR and reloads BIND only on change. **Caveat:** `gen-zones.py` emits only `A` and splits on dots for `/24 in-addr.arpa` — it is IPv4-only. A malformed or IPv6 row would break rendering, so allocation is IPv4-constrained and a pre-deploy zone validation gate is added.

DHCP — a genuine gap NEW

Appliers exist for DNS, AD, Pi-hole — **none for UniFi/DHCP**. `dhcp_ranges` is (once imported, G7) a mirror of the controller. Writing `assignment='dhcp_reservation'` records intent that currently goes nowhere.

The DHCP fork — with a lease guardrail

Provisioned hosts get a **static** address, so they don't strictly need a reservation to resolve. But the review flagged a real hazard: hosts install over DHCP, and `sor_reconcile_discovery.py` can mint a *new* entity for an adopted address not already owned — so a stale lease or a decommissioned host can create ghost identities or collide with controller state.

OPTION	WHAT IT MEANS	REQUIRED GUARDRAIL
A · Static + DNS only `recommended start`	Allocator assigns static IPs; DNS renders automatically; reservations stay a UniFi concern.	UniFi state import (G7) + a **lease-quarantine** rule so the reconciler will not adopt an address that provisioning owns or that belongs to a retired host (tombstone check).
B · SoR-owned reservations	New `sor_apply_unifi.py` pushes reservations so the SoR is authoritative for DHCP too.	UniFi controller API creds + reconcile logic. A project in itself.

Recommendation unchanged: **A** to start (now with the quarantine guardrail and the G7 import as its floor), **B** as a scoped post-freeze follow-on.

08 The lifecycle subsystems

What each part builds and reuses — now including the edges the first draft skipped.

8.1 · Provision — bare metal to enrolled, *with the host actually on its IP*

Extend the working PXE staging into a live, one-motion flow. The blocker the review caught: allocating an IP in the SoR does nothing if the host keeps its DHCP lease. So provisioning includes a **host network cutover** — an idempotent, per-distro step (preseed/late-command, cloud-init, AutoYaST script) that: selects the right interface by MAC, writes the allocated static IP + gateway + DNS resolvers, **confirms connectivity**, and **falls back to DHCP on failure** rather than stranding the box. Only after the host answers on its allocated address is the allocation committed as authoritative. Going live still requires the deliberate DHCP/PXE switch on benzene or UniFi — a conscious operator action, never a side effect.

8.2 · Enroll — one protocol, real endpoints

Today two mechanisms contradict each other: `fleet-lib.sh::sign_node_cert()` generates the private key *on xenon* and stages it over HTTP, while `solari-enroll.sh` expects the host to generate its own CSR + token — and there is **no POST submission endpoint** (`provisioning.php` is GET-only; `enrollments_mut.php` only approves/rejects). Rev 2 picks **one** protocol and specifies it:

- **Host-generated key, always.** The private key is born on the target and never leaves it. Server-side key generation is removed.
- **New endpoints:** `POST /api/enrollments` (submit CSR + enrollment token → pending), `GET /api/enrollments/{id}` (poll status), and the existing approve/reject transitioning pending → signed. A real `enrollment_tokens` table (bound to a pending host + its function tags, single-use, expiring) replaces the client-side fake "issue token" button.
- **One state machine:** `token-issued` → `csr-submitted` → `approved` → `cert-delivered` → `enrolled`, each transition an auditable row.

8.3 · Configure — baseline + role bundles (after a trust refactor)

Generate a dynamic Ansible inventory from the SoR: tags → groups, entity row → `host_vars`. The existing roles are the foundation, but "apply as-is" was overstated — two things must be refactored *first*: the `solarinet-client` role copies a control-node-held private key (incompatible with §8.2's host-key rule), and `ansible.cfg` disables SSH host-key checking (a first-converge MITM exposure). Rev 2 refactors enrollment out of the role and pins host keys (from the enrollment cert / a known-hosts seed) before declaring reuse. New per-tag role bundles then layer the function's packages + config on the shared `common-stack`. `shared-home` and `directory-join` become baseline toggles.

8.4 · Maintain — the LSV lifecycle, made reproducible

The largest new capability, and the one the review pressed hardest on. Six parts:

- **Reproducible "latest stable."** "Latest" is defined per distro by an **immutable manifest** promoted from a repository snapshot — the *same* manifest flows through every ring, so ring 3 can't silently get different packages than the canary. Not a live `apt upgrade` whose result depends on when it ran.

- **Version tracking** — installed vs. manifest, per host, per component (OS set, role bundles, `solariClient`/`solariMonitor`). The control side `/api/builds` never had.
- **Health-gated rings** — canary → ring 1 → ring 2 → critical. A ring advances only when its hosts pass **tag-specific pre/post health probes** (a database answers queries, not merely "package installed"), with quorum for clustered services.
- **Offline & deadline policy** — each run snapshots its eligible population; unreachable hosts get a timeout + waiver + catch-up rule so one dark host neither stalls the ring forever nor is silently skipped and left unpatched.
- **Dry-run & diff** — Ansible check mode reports likely deltas first, with the honest caveat that check mode can't fully predict scriptlets/restarts/migrations; irreversible steps (schema, data) are gated behind an explicit backup/snapshot.
- **Rollback / roll-forward** — where the package manager supports history (zypper, dnf), roll back; where it does not (Debian), the manifest defines a tested roll-*forward* and a workload snapshot taken before the run. Reboot requirements are modeled, not assumed away.

> POLICY GUARDRAIL
> LSV is the default, never a mandate: per-tag **pins/holds**, a global freeze switch, rings that contain a bad manifest to one canary, and mandatory dry-run. A database can deliberately lag the OS train.

8.5 · Observe — orchestrate & watch

The dashboard renders convergence, drift, ring/health status, and run history, and offers enqueue actions. It writes a run row and reads results; it never executes (§11).

8.6 · Retire — the terminal workflow NEW

Omitted in Rev 1 despite `solariCtl.c` already exposing `DECOMMISSION` / `RETIRE`. A retired host must not linger with a valid cert, an owned IP, DNS aliases, secrets, or update eligibility — and must not be *resurrected* by the discovery reconciler. The flow: quarantine → revoke cert → remove secrets → release IP + DNS/DHCP → drop from inventory/rings → write a **discovery tombstone** so `sor_reconcile_discovery.py` won't re-adopt the address → record wipe evidence. This reuses the existing tombstone lifecycle rather than inventing one.

8.7 · Re-tag — changing a host's function NEW

Tags drive network, packages, DNS, and config, so changing `database` → `web` is a migration, not an edit. The workflow: preflight (is the target tag compatible? does it force an IP/VLAN move?) → drain (stop/relocate services) → network cutover + DNS change if the home network changes → converge the new bundle → **remove** the old bundle's packages/secrets (no stale privileged software) → rollback on failure. Incompatible tag combinations are rejected at tag time (§5).

09 Dashboard surface

The single Provisioning page becomes three linked pages. Wireframes are structural, not final visuals.



Provisioning (act on one host, incl. retire) · Build Management (define the system + manifests) · Updates (operate the fleet over time). Each evolves today's Provisioning component.

10 Data-model changes

Normalized to carry the lifecycle — following the incremental-migration + CDC pattern. Names illustrative.

TABLE	PURPOSE	NOTES
`functions`	Closed taxonomy: tag → home network, criticality, description.	Seed data; the semantic spine.
`entity_functions`	Host↔tags, with a `primary_function` flag + incompatibility rules.	Drives groups, allocation, precedence.
`package_bundles`	Per-tag logical package list; reuses `common-stack` name map.	Unions atop shared core.
`function_subnet_map`	Tag → network/subnet the allocator draws from.	Validated; editable in Build Mgmt.
`enrollment_tokens`	Single-use, expiring token bound to a pending host + tags.	Replaces the client-side fake (§8.2).
`ip_allocations`	Allocation lease + the enforced active-address uniqueness (§6).	Atomic claim; reserved-addr rules.
`release_manifests`	Immutable per-distro package/version manifest (a snapshot).	The reproducible "latest stable."
`host_versions`	Installed vs. manifest per host, per component.	Control side of `/api/builds`.

TABLE	PURPOSE	NOTES
`update_rings`	Ring *definition* only (order, cadence).	Split from membership + gate state.
`ring_membership`	Host↔ring, with pin/hold flags.	Was conflated in Rev 1.
`runs / run_attempts / run_targets`	Intent (immutable params + inventory snapshot id) · each execution attempt · per-host result.	Leases, heartbeat, mutex, cancel, timeout (§11).
`health_gates`	Per-tag pre/post probe definitions + quorum.	Gates ring promotion.

Each mutable table gets CDC triggers into `sor_outbox` like the existing seven, so appliers and the dashboard see changes through the same MQ path. The repository stream is `netdb/sor/migrations/` (`001, 014, 015, 020–027`): migration `021` is live; `024–027` are candidate additions and have not been applied to the live SoR.

> **CORRECTION FROM REV 1**
> Rev 1 told us to fix a "duplicate-016" while here. That collision is in `db/migrations/` — the *monitoring* DB — not the SoR stream, which has no 016 at all. It is a separate hygiene item with its own compatibility check, removed from this plan's scope.

11 Execution engine

How intent becomes an Ansible run — with the real topology and crash/concurrency safety.

1. **Enqueue on xenon.** The dashboard calls a `solariCtl` verb (`PROVISION_RUN`, `CONVERGE`, `UPDATE_RING`, `RETIRE`) — which runs *inside solariServer on xenon*, not on benzene. It writes an immutable `runs` row (params + a captured inventory-snapshot id) and signals the bus. No SSH, no CA key, no shell in PHP.
2. **Consume on benzene.** A run-queue daemon on benzene **atomically claims** a run (lease + heartbeat), takes a **per-host mutex** so two runs can't mutate one host, regenerates the inventory from the snapshot id (so a mid-flight tag change can't alter a running job), and invokes `ansible-playbook` with the right `--limit` and tags.
3. **Stream + record.** Output streams to a per-run log the dashboard already tails; on completion the daemon writes status + diff to `run_attempts`/`run_targets`; ring gates advance from health checks, not mere exit codes.
4. **Recover.** A dead daemon's lease expires; the run is resumable or explicitly failed — never silently orphaned or blindly re-run.

WHY NOT JUST COPY THE SORSYNC PATTERN

The `sorsync` appliers are idempotent renderers — re-running re-emits current state harmlessly. An Ansible/package run is long-lived and *partially destructive*; re-running blindly can double-apply or corrupt. So the runner is a **workflow state machine with resumability and per-target reconciliation**, not another renderer. It borrows sorsync's transactional-outbox *enqueue*, not its at-least-once re-execution.

WHY NOT AWX / SEMAPHORE

A full Ansible controller is more service to run, secure, and patch — on the ops hub. The run-queue abstraction is the seam to swap AWX behind later if scale ever demands it; it doesn't demand it now.

12 Security & supply chain

A fleet-wide "install and patch everything" system is a high-value target. The review sharpened where the real risk sits.

No execution surface in PHP

The dashboard enqueues and reads; execution lives on benzene. A compromised dashboard cannot SSH the fleet or sign a cert.

The real top risk: benzene + supply chain

benzene holds fleet SSH + update authority, and `base-os` grants fleet-wide `NOPASSWD:ALL`. Its compromise, or a poisoned artifact, is fleet root — a bad update can pass a naïve canary. Mitigations below are therefore mandatory, not optional.

Signed, immutable artifacts

Binaries and manifests are checksum+signature verified before use — including the firstboot fetch, which currently pulls over HTTP unverified. No unauthenticated mutable artifact enters the pipeline.

Segmented runner credentials

Not one omnipotent key: scope runner credentials by host group / ring, pin SSH host identities (no `host_key_checking=False`), and restrict which playbooks/vars a run may invoke.

Secret custody

Secrets via a vault identity with rotation — never `--extra-vars` on a command line (leaks via process inspection), never in `group_vars`, git, or a run log. Logs are redacted; `no_log` on sensitive tasks.

Certs, provenance, tombstones

Host-generated CSR → internal CA; the provisioner is a registered `sources` row; retirement revokes the cert and tombstones the host so it can't be silently re-adopted (§8.6).

13 Phased delivery roadmap

Re-ordered so no phase secretly depends on a later one. Each phase carries the minimal slice of token API, queue, and UI it needs. Phase 0 implementation is underway under the project authorization; later phases remain gated on the preceding acceptance criteria.

0

Foundations & contract

Migration `021` is deployed on the Cesium SoR and replicated to Benzene: the seven function tags are seeded, and live IP uniqueness and primary-address ownership are database-enforced. Candidate migrations add CDC and an explicit allocator-approval gate; the **UniFi network import** (G7) records controller facts without making them eligible for static assignment. No host behavior yet.

✓ **Accept:** taxonomy seeded; migrations apply clean w/ CDC; UniFi import populates real subnets + `dhcp_ranges` with provenance/freshness; allocator dry-run picks correct, reserved-safe addresses in a unit test.

1

Provision spine + enrollment

One enrollment protocol with real endpoints + `enrollment_tokens` (§8.2). Firstboot → CSR → SoR registration → atomic allocation → **host network cutover** with connectivity check. DNS falls out.

✓ **Accept:** a test host PXE-installs, enrolls (host-born key), lands in the SoR with a function-correct static IP *configured on the host*, resolves forward+reverse — no manual DB edit, cutover verified, DHCP-fallback exercised.

2

Configure via inventory + run queue

Trust refactor (host-key pinning, enrollment out of the role). SoR → dynamic inventory. Minimal run-queue daemon + `runs` table + a dashboard enqueue slice. Baseline + role bundle for the host's tags.

✓ **Accept:** tagging a host `database` and clicking converge runs through the queue (leased, mutexed, snapshotted), installs the bundle + baseline, observed green — driven from the dashboard, no direct SSH.

3

LSV lifecycle + Updates page

Manifests, version tracking, health-gated rings, offline/deadline policy, dry-run, rollback/roll-forward, schedule + one-click — with the Updates UI slice that operates it.

✓ **Accept:** a canary dry-runs, applies, passes tag health probes, and promotes ring-by-ring; a forced-bad manifest halts at canary and rolls back/forward; an offline host is waived not stalled; convergence view accurate.

4

Lifecycle edges

Retirement/decommission (cert revoke, IP/DNS release, tombstone) and function re-tagging (drain, cutover, bundle swap, cleanup). Build Management page complete.

✓ **Accept:** a retired host loses cert/IP/DNS/eligibility and is not re-adopted by discovery; a re-tag migrates network+packages+secrets cleanly with rollback.

5

Hardening & verification gate

Supply-chain signing end-to-end, secret custody/rotation, segmented creds, full run history/audit, and the §15 verification suite as a gate. Cross-lab review.

✓ **Accept:** all three pages drive the pipeline; every action a replayable run row; artifacts signed; verification suite green; independent review dispositioned.

6

Follow-ons (post-freeze)

SoR-owned DHCP applier (Option B), static-musl matrix for appliances, IPv6 allocation + AAAA/ip6.arpa rendering, NAS artifact cache.

✓ **Accept:** scoped separately; each depends on a blocker outside this plan; none gate the core lifecycle.

14 Risks & open decisions

What's resolved in Rev 2, and the calls that remain yours.

#	DECISION / RISK	STATUS & RECOMMENDATION
1	**DHCP ownership** — static+DNS-only vs SoR-owned reservations.	Option A now (with lease-quarantine + G7 import); B post-freeze. `recommend`
2	**LSV aggressiveness.**	Manifests + rings + health gates + per-tag pins + freeze, dry-run mandatory. `resolved in §8.4`
3	**Covered footprint** — appliances / IPv6.	glibc Linux + IPv4 now; musl & IPv6 are Phase 6 follow-ons. `scoped`
4	**Execution engine** — custom queue vs AWX.	Custom workflow state machine; AWX is the later swap-in seam. `recommend`
5	**UniFi import fidelity** — how much controller state to mirror.	At least VLANs/subnets/gateways/pools for safe allocation. `your call on depth`
6	**Go-live gating** — the DHCP/PXE switch.	Stays a deliberate, guarded per-segment operator action. `recommend`
7	**Rollback on Debian** — no package history.	Manifest-defined roll-forward + pre-run workload snapshot. `confirm acceptable`

CANDIDATE SCHEMA RECOVERY

The manual rollback script is `netdb/sor/rollback/024-027-down.sql`. Before any live use, stop the UniFi importer, allocator API, and RackWire SoR writer; take a fresh Cesium backup using the backup and restore runbook. The scheduled 2026-10-22 daily archive predates migration 021. At 21:44 UTC, a fresh post-021 dump from the installed runner restored into isolated MariaDB 12.3.2 with networking disabled; checks passed for all 45 tables, four views, 24 triggers, and four taxonomy tables. Take a fresh backup and repeat the isolated restore check immediately before any future live schema action. Read-only parity was verified on 2026-10-22 21:40 UTC: Cesium and Benzene schema metadata fingerprints match, canonicalized table-data SHA-256 digests match across all SoR tables, and Benzene reports running replication threads with zero lag/errors. Recheck parity if source state changes before rollout. Run these read-only checks against the writable primary. Confirm the schema is `sor`, all four counts are exactly `0, 0, 6, 2`, and the observed 022–023 CDC triggers still exist:

```
SELECT DATABASE();

SELECT COUNT(*) AS network_values_to_preserve
FROM networks
WHERE enabled <> 1 OR management IS NOT NULL OR is_default <> 0
   OR allocator_eligible <> 0 OR allocator_approved_by IS NOT NULL
   OR allocator_approved_at IS NOT NULL;

SELECT COUNT(*) AS cable_values_to_preserve
FROM interconnects
WHERE media_standard IS NOT NULL OR jacket_color IS NOT NULL;

SELECT COUNT(*) AS network_candidate_columns
FROM information_schema.COLUMNS
WHERE TABLE_SCHEMA = DATABASE() AND TABLE_NAME = 'networks'
  AND COLUMN_NAME IN ('enabled','management','is_default',
                      'allocator_eligible','allocator_approved_by',
                      'allocator_approved_at');

SELECT COUNT(*) AS cable_candidate_columns
FROM information_schema.COLUMNS
WHERE TABLE_SCHEMA = DATABASE() AND TABLE_NAME = 'interconnects'
  AND COLUMN_NAME IN ('media_standard','jacket_color');

SELECT TRIGGER_NAME FROM information_schema.TRIGGERS
WHERE TRIGGER_SCHEMA = DATABASE()
  AND TRIGGER_NAME IN ('trg_entities_ai','trg_entities_au','trg_entities_ad');
```

Any different count means stop. Then apply `sudo mariadb sor < netdb/sor/rollback/024-027-down.sql` on Cesium only; replication carries the DDL to Benzene. MariaDB DDL autocommits, so this is not transactional. Never delete migration-ledger rows. Verify the candidate columns, allocator index, and 024/026 triggers are absent and the 022–023 triggers and SoR data remain intact. Re-apply the unchanged 024–027 SQL to restore the candidate structure; this cannot restore any dropped data. A disposable MariaDB rollback-and-forward rehearsal now passes in CI. **VERIFIED (2026-10-22):** Cesium/Benzene schema and data parity; repeat if source state changes. The post-021 restore rehearsal passed; take and restore-check a fresh archive immediately before any future live schema action.

NEXT GATE

Status (2026-10-22): Phase 0 is in progress. Migration `021` is live on Cesium and replicated to Benzene. Candidate `024` adds transactional CDC for `sources` and the mutable function taxonomy; scratch insert/update/delete tests pass. Candidate `025` adds a default-off network allocator approval gate and operator/time audit fields, `026` adds CDC for imported VLANs, DHCP ranges, and external references, and `027` preserves RackWire cable standard/jacket details. These candidates are not applied to the live SoR. `deploy/unifi/unifi_network_sor.py` imports validated controller snapshots transactionally. Scratch MariaDB tests cover migrations 021–027, importer idempotency, overlap-triggered approval revocation, stale-row soft deletion, outbox events, and concurrent allocation with a lock-time approval recheck. A fixture built from the observed Default network (`10.0.0.1/8` , VLAN 1; DHCP `10.1.0.26–10.7.161.117`) confirms allocation is denied by default, then chooses `10.0.0.2` after explicit test approval; this broad DHCP range is never treated as allocator-safe automatically. The runtime allocator also checks every live IPv4 prefix before insertion as a final overlap guard. The dashboard's Provisioning panel passes PHP route tests, a real GET against disposable MariaDB, API/JSX suites, and Chromium browser acceptance at 390px, 320px, and desktop widths; it shows network/VLAN/prefix/gateway/DHCP search, readiness counts, and visible reasons. A read-only live UniFi importer dry-run succeeded with a trusted certificate and a process-only connection pin to `10.0.0.1` , preserving `akoria.net` for TLS hostname verification. Read-only Cesium/Benzene parity was verified at 21:40 UTC on 2026-10-22: both live schemas have matching metadata fingerprints across tables, columns, indexes, constraints, foreign keys, triggers, and views, and their canonicalized table-data SHA-256 digests match across all SoR tables. Benzene reported replication I/O and SQL threads running, zero lag, and zero errors. Both live schemas contain 45 base tables, four views, and 24 triggers. The earlier apparent mismatch came from comparing the replica with the 03:32 -0400 Cesium archive, which predates migration 021; the 12:44 -0400 archive is explicitly pre-021 as well. Candidate migrations 024–027 are absent on Cesium. Scratch rollback and forward-recovery are verified. A post-021 backup restore rehearsal passed on Cesium at 21:44 UTC; a fresh archive and restore-check are still required immediately before any future live schema action. Remaining Phase 0 gates are independent cross-lab review, protected importer runtime configuration, target-specific review and approval of migrations 024–027 and the SoR import, followed by the import and read-only verification. The read-only controller dry-run already passed with trusted TLS and a process-only connection pin; it did not write to the controller or SoR. The timer remains disabled until those checks pass. Unit and fixture coverage proves reserved-safe address selection; a live allocator dry-run remains deferred until imported controller facts are available in SoR. No later phase is accepted.

15 Verification & observability

A fleet-mutating system whose defects first appear in production is a system without a test strategy. This is a phase gate, not an afterthought.

Allocator correctness

Property + concurrency tests: no reserved/gateway/broadcast address ever issued; no two live rows share an address under parallel allocation; pool boundaries honored. Runs in CI against a scratch SoR.

Disposable-VM acceptance

The provision→enroll→cutover→converge flow exercised end-to-end on throwaway VMs across the Debian/Ubuntu/openSUSE matrix, per phase gate — not first tried on a real host.

Failure injection

Forced-bad manifest halts at canary; killed runner daemon recovers via lease expiry; offline host is waived; mid-run tag change doesn't alter the running job. Each is a test, not a hope.

Runner observability

Daemon metrics: queue depth, run duration, stuck-run alerts, lease/heartbeat health, bounded+rotated logs, disk retention. The runner watches itself.

Every phase's acceptance criteria (§13) are backed by one of these; "green" means the suite passed, not that a command exited zero.

16 Appendix — key files & endpoints

Provisioning & netboot

deploy/fleet/fleet-provision.sh · fleet-lib.sh
deploy/fleet/netboot/{README.md,ipxe/boot.ipxe}
deploy/fleet/netboot/installers/solari-firstboot.sh
deploy/enrollment/solari-enroll.sh
deploy/fleet/catalog.json

Ansible

deploy/fleet/ansible/{ansible.cfg,playbooks/site.yml}
deploy/fleet/ansible/group_vars/all.yml
deploy/fleet/ansible/roles/{base-os,common-stack,
directory-join,shared-home,solarinet-client}/

SoR / CDC

netdb/sor/schema.sql (views v_dns_hosts…) · seed.py
netdb/sor/migrations/ (021–023 live; 024–027 candidates)
netdb/gen-zones.py (IPv4 A / /24 PTR)
deploy/sorsync/{sor_emitd,sor_apply_dnsd,
sor_reconcile_discovery}.py · sql/{01,02}

Dashboard & bridge

dashboard/public/screens3.jsx (Provisioning)
dashboard/api/routes/{control,provisioning,config}.php
dashboard/api/routes/enrollments_mut.php
dashboard/api/lib/{SolariCtl,Sor}.php
src/server/solariCtl.c (RETIRE/DECOMMISSION)